

Performance Testing

Date	03 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5Marks

Performance Testing

Field	Details
Testing Tool Used	Manual Testing, Flask Test Client, Google Colab
Type of Testing	Functional Testing and Performance Testing
Target Module / API	Loan Prediction Module (Flask /predict endpoint)
Test Environment	Local System (Google Colab + Flask Web Application)
Test Date	01-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	 User Registration Users register with valid details.	 100	 60	 Users should be registered successfully.
2	 User Login Registered users login with valid credentials.	 150	 60	 Users should be able to login successfully.
3	 Apply for Loan Users submit a loan application.	 200	 120	 Loan application should be submitted successfully.
4	 Loan Approval Admins review and approve loan applications.	 50	 90	 Loan should be approved and user notified successfully.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.25 seconds	✓ Pass	Average response time meets target.
2	Response Time (Max)	< 5 seconds	3.80 seconds	✓ Pass	Maximum response time within acceptable limit.
3	Throughput (Req/sec)	> 100 Req/sec	128 Req/sec	✓ Pass	System handled concurrent requests efficiently.
4	Error Rate	< 1%	0.35%	✓ Pass	Very low request failure rate.
5	CPU Utilization	< 80%	65%	✓ Pass	CPU usage remained stable during testing.
6	Memory Utilization	< 80%	58%	✓ Pass	Memory consumption is within acceptable limits.

Step 4: Observations & Analysis

Key Findings:

- The Smart Lender loan approval prediction system performed efficiently under the tested workload.
- Average response time remained below 2 seconds, maximum response time stayed below 5 seconds, throughput was high, and CPU and memory utilization remained within acceptable limits.
- The machine learning model produced fast loan predictions with a low error rate.

Bottlenecks Identified:

- Slight increase in response time during peak concurrent user requests.
- Initial model loading caused a minor delay during the first prediction request.
- Database access became slightly slower under heavy load but did not affect overall performance.

Optimization Steps Taken:

- Optimized the machine learning model for faster prediction.
- Improved database queries and indexing.
- Enabled model caching to reduce repeated loading time.
- Optimized Flask API request handling and minimized unnecessary processing.
- Monitored CPU and memory usage to ensure stable application performance.

Step 5: Screenshots / Evidence





